Name=Atharv Muik Roll no=27 prn=12412094 batch=CSE(AI)-D-2

```
import pandas as pd
import numpy as np
df=pd.read_csv('/house_prices_practice.csv')
```

a. Read data from different formats (like csv, xls)

```
df
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBat
0	1	7	1560	0	1658	1969	
1	2	4	2827	2	1319	2012	
2	3	8	3920	0	841	2010	
3	4	5	3044	0	1058	1998	
4	5	7	801	1	2428	2020	
...	...	...	...	...	...	...	
295	296	1	3495	1	1792	1954	
296	297	5	3438	3	1266	2003	
297	298	6	1992	0	1148	1996	
298	299	3	3722	1	1407	1998	
299	300	9	1667	0	988	1958	

300 rows × 10 columns

b. Find Shape of Data

```
df.shape
```

```
(300, 10)
```

```
df.head()
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	I
0	1	7	1560	0	1658	1969	2	
1	2	4	2827	2	1319	2012	3	
2	3	8	3920	0	841	2010	1	
3	4	5	3044	0	1058	1998	0	
4	5	7	801	1	2428	2020	0	

```
df.tail()
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBat
295	296	1	3495	1	1792	1954	
296	297	5	3438	3	1266	2003	
297	298	6	1992	0	1148	1996	
298	299	3	3722	1	1407	1998	
299	300	9	1667	0	988	1958	

```
df.loc[[2, 5, 10], 'age'] = np.nan
```

c. Find Missing Values

```
df.isnull().sum()
```

	0
Id	0
OverallQual	0
GrLivArea	0
GarageCars	0
TotalBsmtSF	0
YearBuilt	0
FullBath	0
BedroomAbvGr	0
LotArea	0
SalePrice	0
age	300

dtype: int64

```
df.isnull().value_counts().sum()
```

```
np.int64(300)
```

```
pd.Series([1,2,3,'atharv',76.4])
```

	0
0	1
1	2
2	3
3	atharv
4	76.4

dtype: object

```
freinds = {
    "stud_id" : [1,2,3,4,5],
    "name" : ['atharv','sandy','rudr','shiva','tejas'],
    "age":[19,20,21,22,23],
    "gender":['male','male','male','male','male'],
    "dept":['CSE(AI)','AIDS','CSE','CS(SE)','MECH'],
    "marks":[90,92,94,96,98],
    "attendace":[60,70,80,80,100],
    "internship":['nope','google','microsoft','JP morgan','infosys'],
}
```

```
df1=pd.DataFrame(freinds)
```

```
df1
```

	stud_id	name	age	gender	dept	marks	attendace	internship
0	1	atharv	19	male	CSE(AI)	90	60	nope
1	2	sandy	20	male	AIDS	92	70	google
2	3	rudr	21	male	CSE	94	80	microsoft
3	4	shiva	22	male	CS(SE)	96	80	JP morgan
4	5	tejas	23	male	MECH	98	100	infosys

```
df1.dtypes
```

	0
stud_id	int64
name	object
age	int64
gender	object
dept	object
marks	int64
attendace	int64
internship	object

dtype: object

d. Find data type of each column

```
df.dtypes
```

	0
Id	int64
OverallQual	int64
GrLivArea	int64
GarageCars	float64
TotalBsmtSF	int64
YearBuilt	int64
FullBath	int64
BedroomAbvGr	int64
LotArea	int64
SalePrice	int64
age	float64

dtype: object

e. Finding out Zero's

```
df[df==0].sum()
```

	0
Id	0.0
OverallQual	0.0
GrLivArea	0.0
GarageCars	0.0
TotalBsmtSF	0.0
YearBuilt	0.0
FullBath	0.0
BedroomAbvGr	0.0
LotArea	0.0
SalePrice	0.0
age	0.0

dtype: float64

Indexing and selecting data, sort data,

```
df.iloc[5]
```

	5
Id	6.0
OverallQual	10.0
GrLivArea	680.0
GarageCars	1.0
TotalBsmtSF	2480.0
YearBuilt	1950.0
FullBath	2.0
BedroomAbvGr	1.0
LotArea	13081.0
SalePrice	213952.0
age	NaN

dtype: float64

```
df.iloc[:3]
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	I
0	1	7	1560	0	1658	1969	2	
1	2	4	2827	2	1319	2012	3	
2	3	8	3920	0	841	2010	1	

```
df.loc[0:5]
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	I
0	1	7	1560	0	1658	1969	2	
1	2	4	2827	2	1319	2012	3	
2	3	8	3920	0	841	2010	1	
3	4	5	3044	0	1058	1998	0	
4	5	7	801	1	2428	2020	0	
5	6	10	680	1	2480	1950	2	

```
df[['BedroomAbvGr','GarageCars']]
```

	BedroomAbvGr	GarageCars
0	1	0.0
1	4	2.0
2	4	0.0
3	4	0.0
4	1	1.0
...	...	...
295	5	1.0
296	1	3.0
297	1	0.0
298	1	1.0
299	1	0.0

300 rows × 2 columns

```
df[df['GarageCars'] >2]
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBat
18	19	6	3894	3	1060	1968	
22	23	10	2258	3	1199	2015	
29	30	4	3859	3	971	1978	
41	42	2	3610	3	2136	2014	
42	43	10	2000	3	2394	2006	
46	47	2	2390	3	347	2003	
52	53	4	2029	3	2190	1981	
53	54	2	3450	3	1576	1995	
55	56	4	1184	3	602	2017	
59	60	10	899	3	825	2004	
72	73	2	550	3	1831	1950	
75	76	10	1648	3	1954	1969	
76	77	9	2697	3	784	1960	
77	78	9	2861	3	2375	1951	
85	86	8	3550	3	836	2002	
89	90	3	2811	3	366	1979	
92	93	5	3551	3	1469	1958	
98	99	9	1491	3	1841	1953	
100	101	2	3921	3	2061	2007	
105	106	5	3203	3	1760	1991	
109	110	3	1629	3	1930	1966	
111	112	3	2182	3	1058	1974	
114	115	1	1330	3	528	1980	
128	129	7	1581	3	1764	2003	
132	133	3	1619	3	1758	1985	
134	135	2	1059	3	1733	1978	
135	136	10	987	3	1441	1970	

```
df.sort_values('SalePrice', ascending=True)
```

138	139	6	3843	3	524	1960	
139	140	4	1712	3	831	1985	
140	141	10	1203	3	412	2008	
142	143	9	1080	3	858	2003	

144	145	1	3408	3	2437	1973	
	Id	**OverallQual**	**GrLivArea**	**GarageCars**	**TotalBsmtSF**	**YearBuilt**	**FullBat**
145	146	1	2794	2	952	1971	
268	269	1	1095	0	501	1961	
147	148	9	653	3	1873	2005	
19	20	2	627	0	2442	1967	
149	150	9	1496	2	2217	1985	
232	233	2	1325	0	429	1983	
151	152	7	3244	3	2326	2011	
56	57	2	519	2	938	1986	
152	153	6	2856	2	1201	2022	
218	219	2	1253	0	719	1992	
155	156	5	3618	3	309	2022	
...	...	...	...	...	...	...	.
156	157	1	1508	2	560	2022	
225	226	10	3517	2	2165	1962	
164	165	1	2765	3	1956	1988	
166	167	10	2979	3	2243	1952	
166	167	10	2979	2	2243	1952	
88	89	8	3540	2	1903	2023	
167	168	4	2888	3	2356	1993	
238	239	9	3991	2	1983	1988	
170	171	1	2069	2	1870	2022	
189	190	7	3587	3	2072	2008	
182	183	1	3309	3	2151	1957	

300 rows × 11 columns

183	184	7	3587	3	2072	2008	

```
df.sort_values('Id',ascending=False)
```

214	215	9	2130	3	2335	2009	
	Id	**OverallQual**	**GrLivArea**	**GarageCars**	**TotalBsmtSF**	**YearBuilt**	**FullBat**
217	218	7	3046	3	1222	1986	
299	300	9	1667	0.0	988	1958	
216	217	6	3395	3	1011	2011	
298	299	3	3722	1.0	1407	1998	
245	246	8	2392	3	662	2005	
297	298	6	1992	0.0	1148	1996	
246	247	1	3001	3	2188	1999	
296	297	5	3438	3.0	1266	2003	
248	249	4	2739	3	1939	1974	
295	296	1	3495	1.0	1792	1954	
250	251	6	1522	3	996	2011	
...	...	...	...	...	...	...	.
260	261	2	2048	3	1936	1974	
4	5	7	801	1.0	2428	2020	
256	257	6	1111	3	2131	2002	
3	4	5	3044	0.0	1058	1998	
270	271	4	1228	3	1236	1986	
2	3	8	3920	0.0	841	2010	
258	259	6	2518	3	1000	1999	
1	2	4	2827	2.0	1319	2012	
284	285	6	1217	3	963	2008	
0	1	7	1560	0.0	1658	1969	
299	300	6	3188	3	1200	2000	

300 rows × 11 columns

g. Describe attributes of data, checking data types of each column,

```
df.describe()
```

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBu
count	300.000000	300.000000	300.000000	300.000000	300.000000	300.000
mean	150.500000	5.326667	2307.386667	1.330000	1468.796667	1986.163
std	86.746758	2.873001	1042.561303	1.109898	672.333705	21.377
min	1.000000	1.000000	504.000000	0.000000	303.000000	1950.000
25%	75.750000	3.000000	1392.250000	0.000000	903.000000	1967.000
50%	150.500000	5.000000	2265.500000	1.000000	1502.000000	1986.000
75%	225.250000	8.000000	3306.500000	2.000000	2129.500000	2004.250
max	300.000000	10.000000	3998.000000	3.000000	2492.000000	2023.000

```
df['YearBuilt'].value_counts()
```

YearBuilt	count
2008	9
2011	8
1974	8
1986	7
1975	7
...	...
1999	1
1976	1
2007	1
2013	1
2001	1

73 rows × 1 columns

dtype: int64

h. counting unique values of data, format of each column, converting variable data type (e.g. from long to short, vice versa)

```
df['GarageCars'] = df['GarageCars'].astype('float')
```

```
df.dtypes
```

	0
Id	int64
OverallQual	int64
GrLivArea	int64
GarageCars	float64
TotalBsmtSF	int64
YearBuilt	int64
FullBath	int64
BedroomAbvGr	int64
LotArea	int64
SalePrice	int64
age	float64

dtype: object

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Id            300 non-null    int64
 1   OverallQual   300 non-null    int64
 2   GrLivArea     300 non-null    int64
 3   GarageCars    300 non-null    float64
 4   TotalBsmtSF   300 non-null    int64
 5   YearBuilt     300 non-null    int64
 6   FullBath      300 non-null    int64
 7   BedroomAbvGr  300 non-null    int64
 8   LotArea       300 non-null    int64
 9   SalePrice     300 non-null    int64
 10  age           0 non-null      float64
dtypes: float64(2), int64(9)
memory usage: 25.9 KB
```